



Universidad de la Frontera

Sistema Inteligente de Gestión de Tráfico con Aprendizaje por Refuerzo

Profesor:

Ricardo Gacitua Bustos

Integrante:

Fidel Barriga, Ingeniería Civil Informática

Asignatura: Sistemas inteligentes

Temuco, 10 de julio de 2026

1. Introducción

El aprendizaje por refuerzo (RL) es un tipo de proceso de machine learning en el que los agentes autónomos aprenden a tomar decisiones interactuando con su entorno (Ph.D. & Kavlakoglu, 2020). Su objetivo es descubrir qué acciones tomar en diferentes situaciones para maximizar una señal de recompensa numérica a lo largo del tiempo. Este enfoque es útil, especialmente, para problemas complejos donde las reglas cambian constantemente.

2. Descripción de conceptos

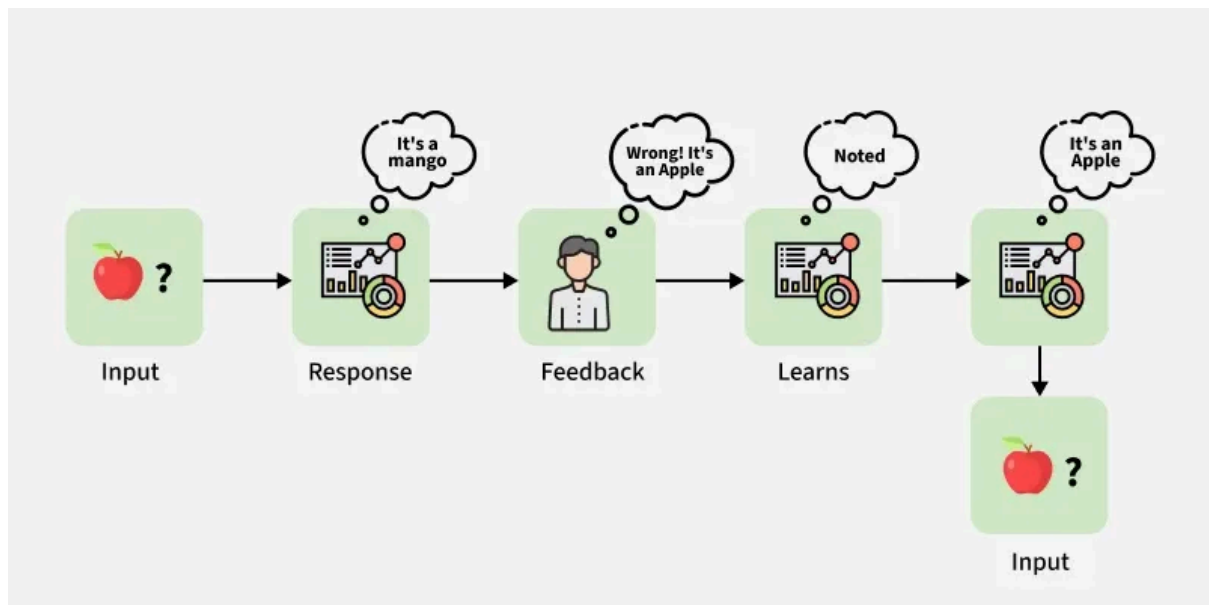
Para modelar un problema utilizando RL es necesario definir los elementos que componen su ecosistema. para el contexto del proyecto se define de la siguiente manera:

- **Agente (Agent):** Corresponde al algoritmo o sistema que toma las decisiones. Su objetivo es aprender a realizar una tarea de la mejor manera posible.
- **Entorno (Environment):** Es el “Espacio” donde se mueve el agente. Corresponde a todo lo que rodea al agente y con lo que puede interactuar
- **Estado (State):** Representa la situación en la que se encuentra el entorno y el agente en un instante de tiempo específico.
- **Acción (Action):** Corresponde a las opciones o movimientos que el agente puede elegir realizar en un estado determinado.
- **Recompensa (Reward):** Es la retroalimentación que el entorno devuelve al agente después de que este realice una acción. Puede ser positiva o negativa dependiendo de cómo se encuentre definido en el entorno.
- **Política (Policy):** Es la estrategia que sigue el agente. La función que le dice que acción debe tomar el agente basándose en el estado en el que se encuentra. Al principio la política suele ser aleatoria, pero con el entrenamiento se vuelve óptima.

3. Explicación del proceso de aprendizaje mediante recompensas

El proceso de aprendizaje de RL se basa en ensayo y error. En cada paso, el agente evalúa el estado actual del entorno y selecciona una acción. En consecuencia, el entorno cambia a un nuevo estado y le entrega una recompensa al agente.

Con todo esto, el agente busca maximizar la suma de recompensas a largo plazo. Al principio del entrenamiento el agente carece de conocimientos, por lo que explora el entorno mediante acciones aleatorias (fase de exploración). A medida que descubre qué acciones le otorgan recompensas positivas y cuales lo penalizan, ajusta su comportamiento para elegir las más beneficiosas.



Representación de aprendizaje en RL

4. Descripción del Algoritmo de RL: Q-Learning

Para efectos de este proyecto se implementó Q-learning. Es un algoritmo basado en valores y fuera de política que encontrará la mejor serie de acciones basándose en el estado actual del agente. La "Q" significa calidad y representa el valor de la acción para maximizar las recompensas futuras (Ali, 2024).

Q-learning funciona creando y actualizando una matriz bidimensional llamada Q-Table, donde las filas representan todos los estados posibles y las columnas las acciones posibles. El algoritmo actualiza los valores de la tabla utilizando la ecuación de Bellman (Ali, 2024).

Actualiza el valor Q del par estado acción actual

Valor Q almacenado de la última vez que realizó esa acción en ese estado

Nuevo valor Q para el estado y acción actual

$$Q_n(s, a) = (1 - \alpha)Q_p(s, a) + \alpha Q_o(s, a)$$

Siendo,

$$Q_o(s, a) = r(s, a) + \gamma \max_{a'} Q(s', a')$$

Ecuación de Bellman

Cada vez que el agente toma una acción, el valor Q para el par (estado, acción) se ajusta hacia la recompensa recibida más el máximo valor Q futuro esperado del siguiente

estado. Para trabajar con los valores continuos de mi entorno (velocidad y densidad) se aplicó una técnica de discretización, reduciendo los infinitos valores de los sensores a 9 estados discretos y manejables para la Q-Table.

5. Caso Práctico de Detección de Anomalías

El caso práctico desarrollado corresponde a la detección de anomalías en el tráfico vehicular. Una anomalía se define aquí como un accidente o bloqueo imprevisto en la vía principal, dicha anomalía provocaría una caída en la velocidad promedio y un aumento en la densidad de tráfico.

El modelo de recompensas se estableció de la siguiente manera:

- **Verdadero Positivo (+10 puntos):** El agente detecta la anomalía y desvía el tráfico exitosamente.
- **Falso Positivo (-5 puntos):** El agente decide desviar el tráfico cuando las condiciones son normales .
- **Falso Negativo (-10 puntos):** Ocurre un accidente y el agente no hace nada.
- **Verdadero Negativo (+5 puntos):** Tráfico fluido y el agente mantiene la vía abierta.

Por medio del entrenamiento, el agente “aprende” a reconocer el patrón del estado anómalo (baja velocidad, alta densidad) y lo asocia con la acción de "Desviar".

6. Visualización y Simulación del Comportamiento del Agente

Para demostrar el funcionamiento del modelo de manera interactiva, se desarrolló una aplicación web con la siguiente arquitectura: un backend en Python (FastAPI) que aloja la lógica del algoritmo Q-Learning, y un frontend en React que provee la interfaz gráfica.

La plataforma permite al usuario iniciar el entrenamiento del agente (1000 episodios) y luego simular el paso del tiempo. Se incorporó una visualización en 2D que representa la autopista vista desde arriba. En esta simulación gráfica, el usuario puede observar en tiempo real los datos de los sensores y cómo el sistema responde físicamente.

7. Conclusiones y Limitaciones de la Propuesta

El Aprendizaje por Refuerzo demuestra ser altamente efectivo para resolver el problema de la gestión de tráfico ante anomalías. En lugar de programar reglas fijas, el agente fue capaz de deducir por sí mismo la política óptima basándose puramente en aprendizaje por medio de recompensas, alcanzando un desempeño casi perfecto tras unos cuantos episodios de entrenamiento.

La limitación principal de esta propuesta recae en la técnica de discretización de los estados. Si bien reducir las variables a 9 estados permitió utilizar el algoritmo clásico de Q-Learning, en un escenario de la vida real con múltiples avenidas, semáforos y miles de vehículos, el número de estados crecería exponencialmente, haciendo que la Q-Table sea imposible de calcular y almacenar en memoria.

Para superar esta limitación, se requeriría transicionar de Q-Learning tradicional a Deep Reinforcement Learning (como Deep Q-Networks o DQN), reemplazando la tabla por una Red Neuronal Artificial capaz de procesar estados continuos de alta complejidad.